

Deep Learning

Transformer

Research Depth and Industry Skills

By:

Dr. Zohair Ahmed



www.youtube.com/@ZohairAI

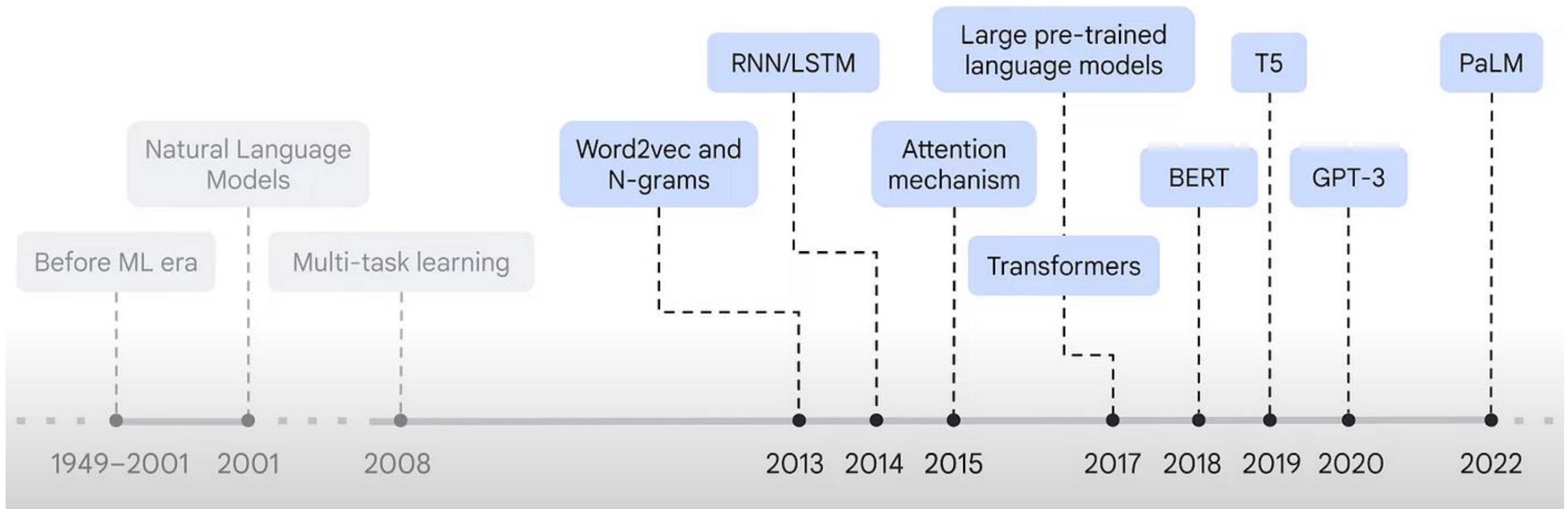
Subscribe



www.begindiscovery.com

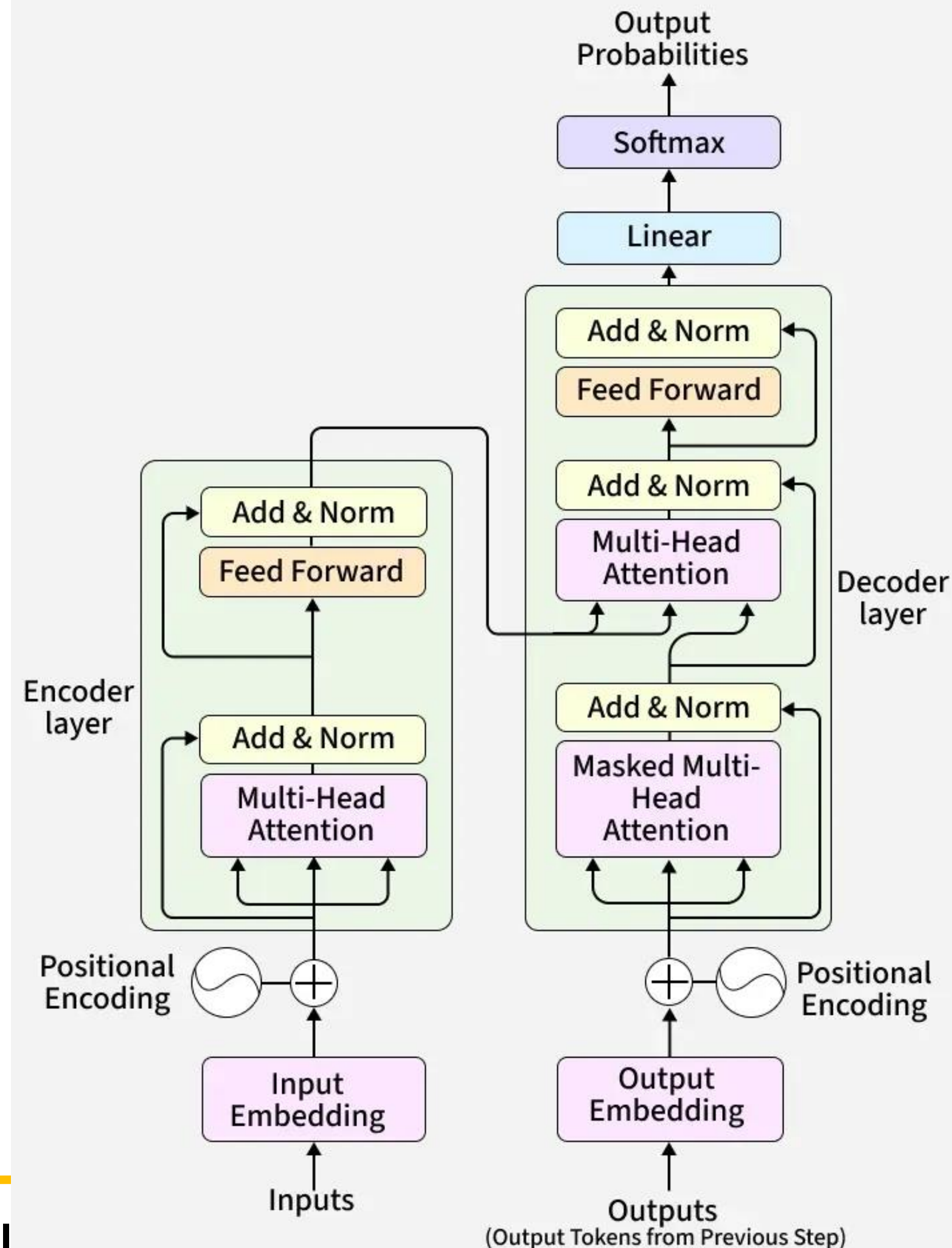
Language Model History

Language modeling history



What is Transformer

- The transformer architecture has revolutionized AI, particularly in NLP.
- It uses a **self-attention** mechanism and **parallel processing** to improve performance.
- The key components also include **multi-head attention** and **positional encoding**.
- Unlike traditional sequence models such as **RNNs** and **LSTMs**, transformers leverage a self-attention mechanism that enables efficient parallelization and improved performance.



Transformer Architecture?

- The **transformer architecture** is a deep learning model introduced in the paper [Attention Is All You Need](#) by Vaswani et al. (2017).
- It eliminates the need for recurrence by using self-attention and positional encoding, making it highly effective for sequence-to-sequence tasks such as language translation and text generation.

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

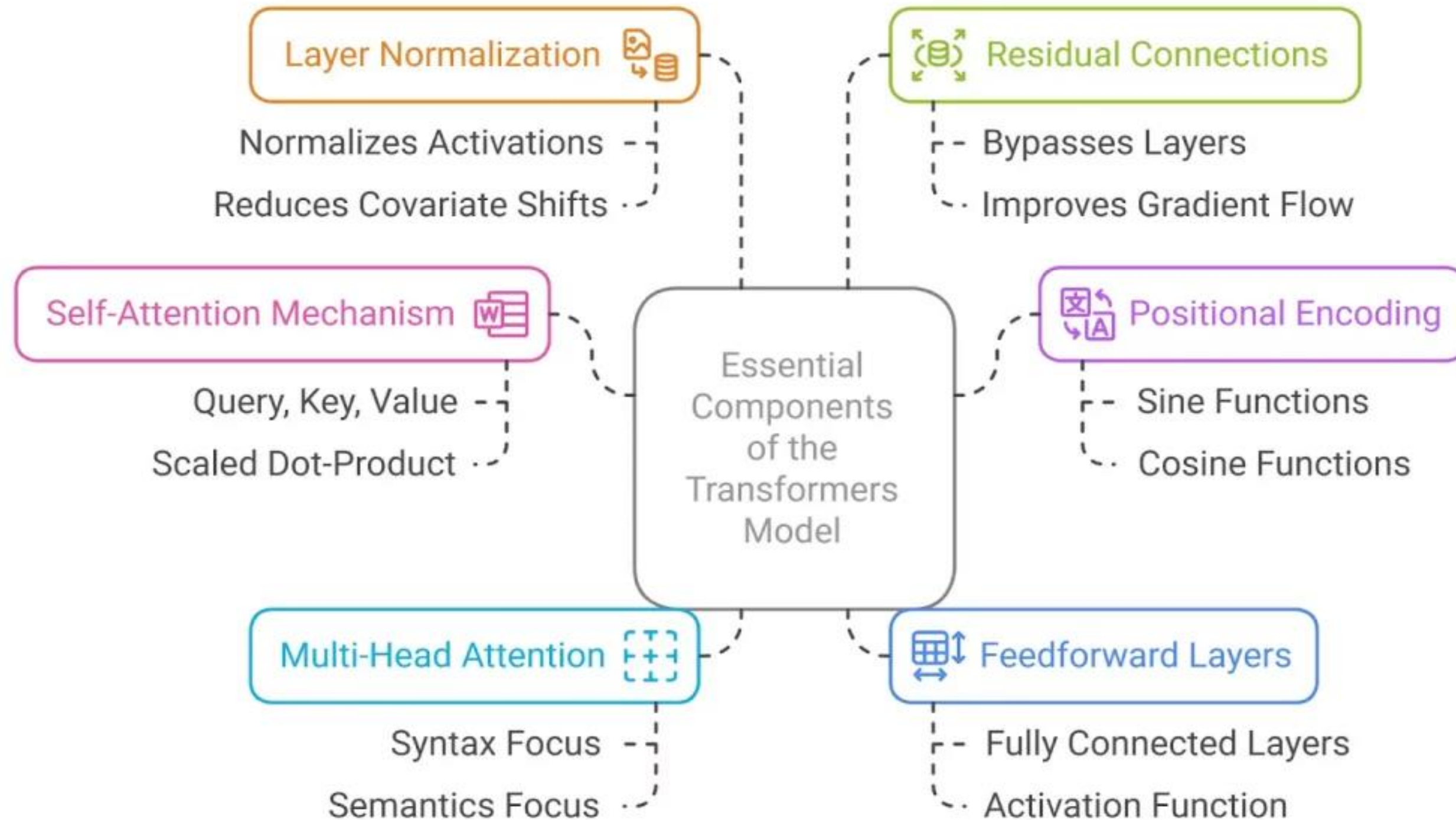
Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.0 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature.



Essential Component of Transformer



1. Self-Attention Mechanism

- The self-attention mechanism allows the model to consider all words in a sequence simultaneously, focusing on the most relevant ones regardless of position.
- Unlike sequential RNNs, it processes relationships between all words at once.
- Each word is represented through **Query (Q)**, **Key (K)**, and **Value (V)** matrices.
- Relevance between words is calculated using the scaled dot-product formula.
- $\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V}$
- For instance, in "The **cat** **sat** on the **mat**," "**cat**" might strongly attend to "**sat**" rather than "**mat**."

2. Positional Encoding

- Since transformers don't process input sequentially, positional encoding preserves word order by adding positional information to word embeddings. This encoding uses sine and cosine functions:
- $PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$ for *even* indices
- $PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$ for *odd* indices
- Without this encoding, sentences like **"He ate the apple"** and **"The apple ate he"** would appear identical to the model.

3. Multi-Head Attention

- This feature applies self-attention multiple times in parallel, with each attention head learning different linguistic patterns.
- Some heads might focus on syntax (subject-verb relationships), while others capture semantics (word meanings).
- These parallel outputs are then concatenated into a unified representation.

4. Feedforward Layers

- Each transformer block contains feedforward neural networks that process attention outputs. These consist of two fully connected layers with an activation function between them:
- $FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$
- These layers enhance feature representation by transforming the attention-weighted input.

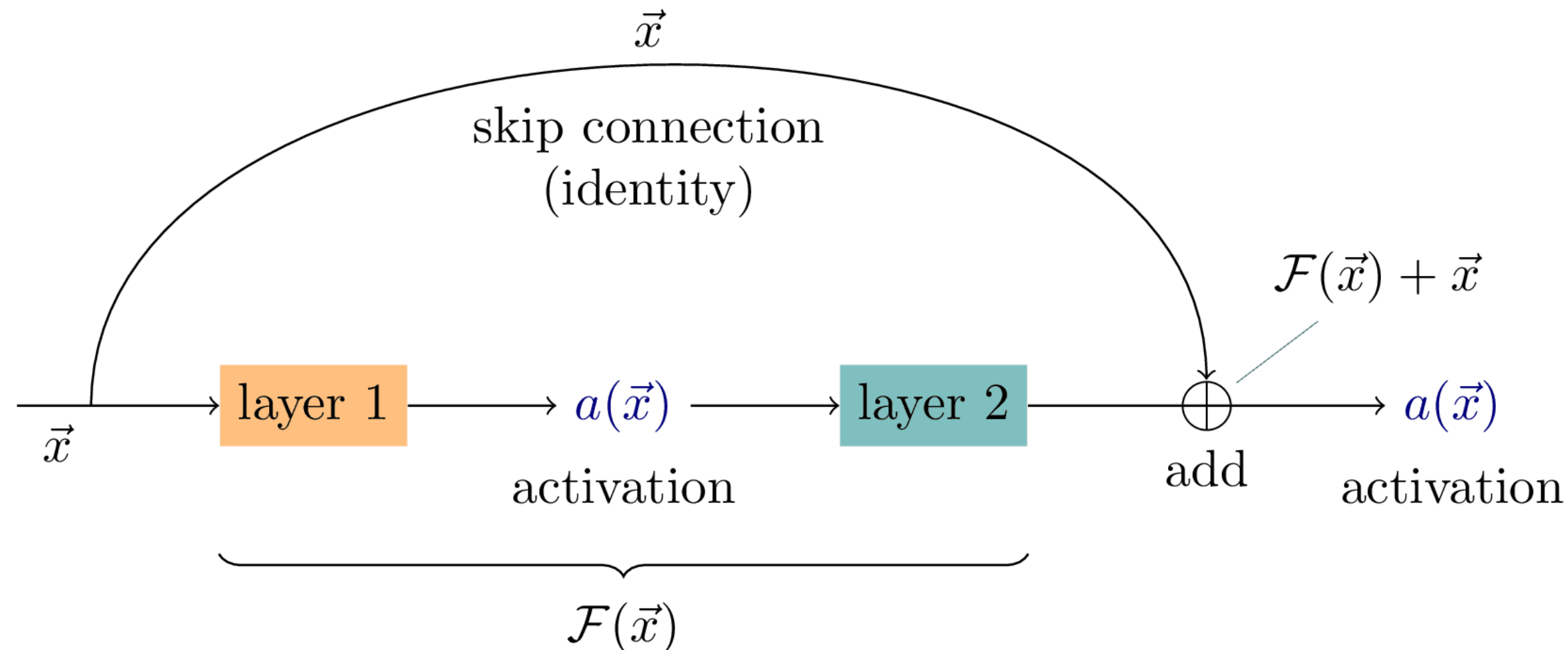
5. Layer Normalization

- Layer normalization stabilizes training by normalizing activations across features, which reduces internal covariate shifts and improves convergence speed.
- During training, this normalization prevents sudden changes in feature magnitudes, making the learning process more consistent.



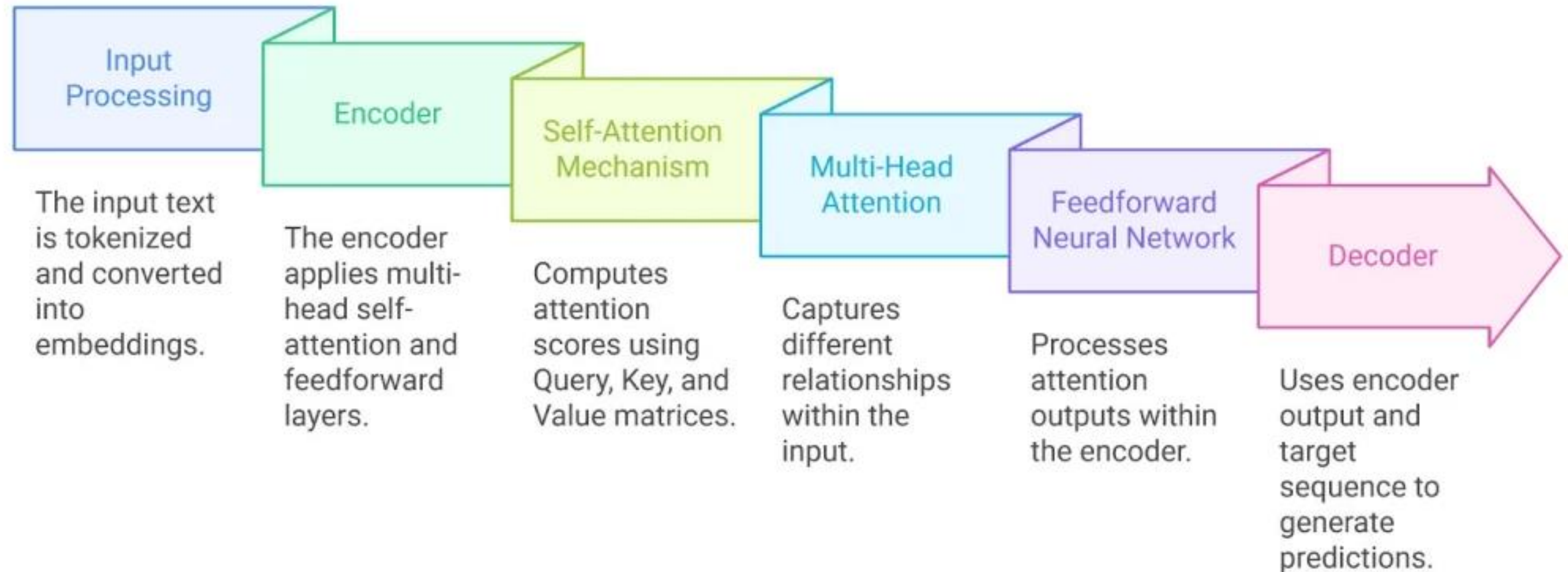
6. Residual Connections

- Transformers implement residual (skip) connections that allow information to bypass multiple layers, improving gradient flow and preventing information loss.
- These connections are especially crucial in deep transformer stacks, where they ensure original information remains intact and help mitigate vanishing gradient problems.



How the Transformers Model Works?

- The **transformer model** consists of an encoder and decoder, both built using multiple layers of self-attention and feedforward networks.



How the Transformers Model Works?

1. Input Processing

- The input text is tokenized and converted into word embeddings.
- Positional encodings are added to maintain word order information.

2. Encoder

- Takes input embeddings and applies **multi-head self-attention**.
- Utilizes **positional encodings** to maintain word order.
- Passes information through **feedforward layers** for processing.

3. Self-Attention Mechanism

- The self-attention mechanism allows each word in a sentence to focus on other relevant words dynamically. The steps include:
- Computing **Query (Q)**, **Key (K)**, and **Value (V)** matrices for each word.
- Generating attention scores using **scaled dot-product attention**.
- Applying **softmax** to normalize attention scores.

- Weighting **value vectors** accordingly and summing them.

4. Multi-Head Attention

- Instead of a single attention mechanism, **multi-head attention** allows the model to capture different relationships within the input.

5. Feedforward Neural Network

- Each encoder layer has a fully connected **feedforward network (FFN)** that processes attention outputs.

6. Decoder

- Receives encoder output along with target sequence.
- Uses **masked self-attention** to prevent looking ahead.
- Combines encoder-decoder attention to refine output predictions.

Step-by-Step Processing

- **Tokenization & Embedding:**

- Words are tokenized:
- ['Transformers', 'are', 'changing', 'AI', '.']
- Each token is converted into a vector representation.

- **Positional Encoding:**

- Encodes the position of words in the sequence.

- **Encoder Self-Attention:**

- The model computes attention weights for each word.
- **Example:** "Transformers" might have high attention on "changing" but less on "AI".

- **Multi-Head Attention:**

- Multiple attention heads capture different linguistic patterns.

- **Decoder Processing:**

- The decoder starts with the <SOS> (Start of Sequence) token.
- It predicts the first word ("Les" for "The Transformers").
- Uses previous predictions iteratively to generate the next word.

- **Output Sentence:**

- The final translated sentence: **"Les Transformers changent l'IA."**

Applications of Transformer Architecture

- The **transformer architecture** is widely used in AI applications, including:
- **Natural Language Processing (NLP):**
 - Powering models like BERT, GPT, and T5.
- **Machine Translation:**
 - Used in Google Translate and similar applications.
- **Text Summarization:**
 - Enabling AI-driven summarization tools.
- **Speech Recognition:**
 - Enhancing voice assistants like Alexa and Siri.
- **Computer Vision:**
 - Applied in vision transformers (ViTs) for image processing.

Advantages of Transformer NN Architecture

- **Parallelization:**
 - Unlike RNNs, transformers process input sequences simultaneously.
- **Long-Range Dependencies:**
 - Effectively captures relationships between distant words.
- **Scalability:**
 - Easily adaptable to larger datasets and more complex tasks.
- **State-of-the-Art Performance:**
 - Outperforms traditional models in NLP and AI applications.

Attention Mechanism



What is Attention

- **Attention means:**
- When generating output, the model decides **which input words are more important** and gives them **higher weight**.
- So instead of treating all words equally, it assigns **attention scores**.
- Example sentence:
- **Input:** I love NLP
- If we want to focus on the word **love**, attention may decide that:
 - "I" is less important
 - "love" is very important
 - "NLP" is also important

- $Attention(Q, K, V) = Softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) V$
 - **Q = Query**
 - **K = Key**
 - **V = Value**
 - d_k = dimension of Key vectors

Why We Need Q, K, V?

- For each word, we create 3 vectors:
- **Query (Q)**
 - "What am I looking for?"
- **Key (K)**
 - "What do I contain?"
- **Value (V)**
 - "What information do I provide?"

Step 1: Create embeddings of input words

- Suppose we have 3 words:
 - $w_1 = I$
 - $w_2 = love$
 - $w_3 = NLP$
- Assume embedding dimension = 2.
- So, embeddings are:
 - $X_1 = [1, 0]$
 - $X_2 = [0, 1]$
 - $X_3 = [1, 1]$
- Now combine into a matrix:
 - $X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$
 - Shape:
 - $X: (3 \times 2)$

Step 2: Define Weight Matrices

- **Define Weight Matrices:** W_Q, W_K, W_V
- We need to compute Q, K, V.
- Assume:

- $W_Q = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$

- $W_K = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$

- $W_V = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$

- All shapes:
- $W_Q, W_K, W_V: (2 \times 2)$

Step 3: Compute Q, K, V

- **Formula:**
- $Q = XW_Q$
- $K = XW_K$
- $V = XW_V$

Step 3A: Compute Q, K, V - Start with Compute Q

- **Compute Q**

- $Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$

- Now multiply row-by-row.

- **Q for word 1 (row 1)**

- $[1, 0] \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$

- Compute first element: $1 \cdot 1 + 0 \cdot 0 = 1$

- Second element: $1 \cdot 1 + 0 \cdot 1 = 1$

- So: $Q_1 = [1, 1]$

- **Q for word 2 (row 2)**

- $[0, 1] \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$

- First element: $0 \cdot 1 + 1 \cdot 0 = 0$

- Second element: $0 \cdot 1 + 1 \cdot 1 = 1$

- So: $Q_2 = [0, 1]$

- **Q for word 3 (row 3)**

- $[1, 1] \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$

- First element: $1 \cdot 1 + 1 \cdot 0 = 1$

- Second element: $1 \cdot 1 + 1 \cdot 1 = 2$

- So: $Q_3 = [1, 2]$

- **Final Q matrix**

- $Q = \begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 2 \end{bmatrix}$



Step 3B: Compute **K**

- **Compute K**

- $K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$

- **K for word 1**

- $[1, 0] \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$

- First element: $1 \cdot 1 + 0 \cdot 1 = 1$

- Second element: $1 \cdot 0 + 0 \cdot 1 = 0$

- **So: $K_1 = [1, 0]$**

- **K for word 2**

- $[0, 1] \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$

- First element: $0 \cdot 1 + 1 \cdot 1 = 1$

- Second element: $0 \cdot 0 + 1 \cdot 1 = 1$

- **So: $K_2 = [1, 1]$**

- **K for word 3**

- $[1, 1] \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$

- First element: $1 \cdot 1 + 1 \cdot 1 = 2$

- Second element: $1 \cdot 0 + 1 \cdot 1 = 1$

- **So: $K_3 = [2, 1]$**

- **Final K matrix**

- $K = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 2 & 1 \end{bmatrix}$

Step 3C: Compute **V**

- **Compute V**

- $V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$

- **V for word 1**

- $[1, 0] \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$

- First element: $1 \cdot 1 + 0 \cdot 0 = 1$

- Second element: $1 \cdot 2 + 0 \cdot 1 = 2$

- **So: $V_1 = [1, 2]$**

- **V for word 2**

- $[0, 1] \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$

- First element: $0 \cdot 1 + 1 \cdot 0 = 0$

- Second element: $0 \cdot 2 + 1 \cdot 1 = 1$

- **So: $V_2 = [0, 1]$**

- **V for word 3**

- $[1, 1] \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$

- First element: $1 \cdot 1 + 1 \cdot 0 = 1$

- Second element: $1 \cdot 2 + 1 \cdot 1 = 3$

- **So: $V_3 = [1, 3]$**

- **Final V matrix**

- $V = \begin{bmatrix} 1 & 2 \\ 0 & 1 \\ 1 & 3 \end{bmatrix}$



Step 4: Compute Attention Scores

- The score formula:

- $Score = QK^T$

- First find:

- $K^T = \begin{bmatrix} 1 & 1 & 2 \\ 0 & 1 & 1 \end{bmatrix}$

- Now multiply:

- $QK^T = \begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & 1 & 2 \\ 0 & 1 & 1 \end{bmatrix}$

- This gives a 3×3 matrix.

Compute each element one by one

Row 1 of Q is [1,1]

- **Score(1,1):** $[1, 1] \cdot [1, 0] = 1 \cdot 1 + 1 \cdot 0 = 1$
- **Score(1,2):** $[1, 1] \cdot [1, 1] = 1 \cdot 1 + 1 \cdot 1 = 2$
- **Score(1,3):** $[1, 1] \cdot [2, 1] = 1 \cdot 2 + 1 \cdot 1 = 3$
- **So first row = [1,2,3]**

Row 2 of Q is [0,1]

- **Score(2,1):** $[0, 1] \cdot [1, 0] = 0 \cdot 1 + 1 \cdot 0 = 0$
- **Score(2,2):** $[0, 1] \cdot [1, 1] = 0 \cdot 1 + 1 \cdot 1 = 1$
- **Score(2,3):** $[0, 1] \cdot [2, 1] = 0 \cdot 2 + 1 \cdot 1 = 1$
- **So second row = [0,1,1]**

Row 3 of Q is [1,2]

- **Score(3,1):** $[1, 2] \cdot [1, 0] = 1 \cdot 1 + 2 \cdot 0 = 1$
- **Score(3,2):** $[1, 2] \cdot [1, 1] = 1 \cdot 1 + 2 \cdot 1 = 3$
- **Score(3,3):** $[1, 2] \cdot [2, 1] = 1 \cdot 2 + 2 \cdot 1 = 4$
- **So third row = [1,3,4]**
- **Final Score Matrix**

- $Score = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 1 \\ 1 & 3 & 4 \end{bmatrix}$

Step 5: Scale the Scores

- We divide by: $\sqrt{d_k}$
- Here Key dimension $d_k = 2$
- So: $\sqrt{2} \approx 1.414$
- Now scaled score:
- $ScaledScore = \frac{Score}{1.414}$
- Compute each element:
- **Row 1:**
 - $1 / 1.414 = 0.707$
 - $2 / 1.414 = 1.414$
 - $3 / 1.414 = 2.121$
- **Row 2:**
 - $0 / 1.414 = 0$
 - $1 / 1.414 = 0.707$
 - $1 / 1.414 = 0.707$
- **Row 3:**
 - $1 / 1.414 = 0.707$
 - $3 / 1.414 = 2.121$
 - $4 / 1.414 = 2.828$
- So:
- $ScaledScore = \begin{bmatrix} 0.707 & 1.414 & 2.121 \\ 0 & 0.707 & 0.707 \\ 0.707 & 2.121 & 2.828 \end{bmatrix}$

Step 6: Apply Softmax Row-Wise

- $\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$
- We apply softmax **on each row** because each word attends to all words.
- **Softmax Row 1: [0.707, 1.414, 2.121]**
- **Compute exponentials:**
 - $e^{0.707} = 2.028$
 - $e^{1.414} = 4.113$
 - $e^{2.121} = 8.340$
- **Sum:** $2.028 + 4.113 + 8.340 = 14.481$
- **Now divide:**
 - $a_{11} = 2.028/14.481 = 0.140$
 - $a_{12} = 4.113/14.481 = 0.284$
 - $a_{13} = 8.340/14.481 = 0.576$
- **So row 1 attention weights: [0.140, 0.284, 0.576]**
- **Softmax Row 2: [0, 0.707, 0.707]**
- **Exponentials:**
 - $e^0 = 1$
 - $e^{0.707} = 2.028$
 - $e^{0.707} = 2.028$
- **Sum:** $1 + 2.028 + 2.028 = 5.056$
- **Weights:**
 - $a_{21} = 1/5.056 = 0.198$
 - $a_{22} = 2.028/5.056 = 0.401$
 - $a_{23} = 2.028/5.056 = 0.401$
- **So row 2: [0.198, 0.401, 0.401]**
- **Softmax Row 3: [0.707, 2.121, 2.828]**
- **Exponentials:**
 - $e^{0.707} = 2.028$
 - $e^{2.121} = 8.340$
 - $e^{2.828} = 16.930$
- **Sum:** $2.028 + 8.340 + 16.930 = 27.298$
- **Weights:**
 - $a_{31} = 2.028/27.298 = 0.074$
 - $a_{32} = 8.340/27.298 = 0.305$
 - $a_{33} = 16.930/27.298 = 0.621$
- **So row 3: [0.074, 0.305, 0.621]**



Step 6: Apply Softmax Row-Wise

- **Final Attention Weight Matrix**

- **This is called:**

- $A = \text{softmax}(\text{ScaledScore})$

- $A = \begin{bmatrix} 0.140 & 0.284 & 0.576 \\ 0.198 & 0.401 & 0.401 \\ 0.074 & 0.305 & 0.621 \end{bmatrix}$

- **Shape:**

- $A: (3 \times 3)$

Step 7: Multiply Attention Weights with V

- Final output formula:

- $Output = AV$

- We have:

- $V = \begin{bmatrix} 1 & 2 \\ 0 & 1 \\ 1 & 3 \end{bmatrix}$

- Now compute:

- $Output = \begin{bmatrix} 0.140 & 0.284 & 0.576 \\ 0.198 & 0.401 & 0.401 \\ 0.074 & 0.305 & 0.621 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 0 & 1 \\ 1 & 3 \end{bmatrix}$

- Output will be 3×2

Step 7: Multiply Attention Weights with V

- **Compute Output Row-by-Row**
- **Output Row 1**
- Row 1 of A:
- $[0.140, 0.284, 0.576]$
- Multiply with V:
- $Output_1 = 0.140V_1 + 0.284V_2 + 0.576V_3$
- Where:
 - $V_1 = [1, 2]$
 - $V_2 = [0, 1]$
 - $V_3 = [1, 3]$
- **Compute first element of Output_1**
 - $0.140(1) + 0.284(0) + 0.576(1)$
 - $= 0.140 + 0 + 0.576 = 0.716$
- **Compute second element of Output_1**
 - $0.140(2) + 0.284(1) + 0.576(3)$
 - $= 0.280 + 0.284 + 1.728 = 2.292$
- **So: $Output_1 = [0.716, 2.292]$**
- **Output Row 2**
- **Row 2 of A:**
- $[0.198, 0.401, 0.401]$
- $Output_2 = 0.198V_1 + 0.401V_2 + 0.401V_3$
- **First element**
 - $0.198(1) + 0.401(0) + 0.401(1)$
 - $= 0.198 + 0 + 0.401 = 0.599$
- **Second element**
 - $0.198(2) + 0.401(1) + 0.401(3)$
 - $= 0.396 + 0.401 + 1.203 = 2.000$
- **So: $Output_2 = [0.599, 2.000]$**
- **Output Row 3**
- **Row 3 of A:**
- $[0.074, 0.305, 0.621]$
- $Output_3 = 0.074V_1 + 0.305V_2 + 0.621V_3$
- **First element**
 - $0.074(1) + 0.305(0) + 0.621(1)$
 - $= 0.074 + 0 + 0.621 = 0.695$
- **Second element**
 - $0.074(2) + 0.305(1) + 0.621(3)$
 - $= 0.148 + 0.305 + 1.863 = 2.316$
- **So: $Output_3 = [0.695, 2.316]$**
- **Final Attention Output Matrix**
- $Output = \begin{bmatrix} 0.716 & 2.292 \\ 0.599 & 2.000 \\ 0.695 & 2.316 \end{bmatrix}$
- This is the final output of attention.

What Does This Output Mean?

- Each row is the **new representation** of a word after attention.
- So:
- Output row 1 = new embedding of word "I"
- Output row 2 = new embedding of word "love"
- Output row 3 = new embedding of word "NLP"
- Each one now contains **information mixed from all words**.

Important Interpretation of Attention Weights

- **Attention matrix:**

$$A = \begin{bmatrix} 0.140 & 0.284 & 0.576 \\ 0.198 & 0.401 & 0.401 \\ 0.074 & 0.305 & 0.621 \end{bmatrix}$$

- Row 1: **"I"** attends most to **NLP** (0.576)
- Row 2: **"love"** attends equally to **love** and **NLP** (0.401, 0.401)
- Row 3: **"NLP"** attends most to **NLP** itself (0.621)



What Does This Output Mean?

- **Rows = Query words** (who is attending)
- **Columns = Key words** (who is being attended to)
- **Input words order:**
- 1 = **I**, 2 = **Love**, 3 = **NLP**

$$A = \begin{bmatrix} 0.140 & 0.284 & 0.576 \\ 0.198 & 0.401 & 0.401 \\ 0.074 & 0.305 & 0.621 \end{bmatrix}$$

Query \ Key	I (col 1)	Love (col 2)	NLP (col 3)
I (row 1)	$A(1,1) = 0.140$	$A(1,2) = 0.284$	$A(1,3) = 0.576$
Love (row 2)	$A(2,1) = 0.198$	$A(2,2) = 0.401$	$A(2,3) = 0.401$
NLP (row 3)	$A(3,1) = 0.074$	$A(3,2) = 0.305$	$A(3,3) = 0.621$

Summary of All Steps

- **Input embeddings**

- X

- **Compute**

- $Q = XW_Q, K = XW_K, V = XW_V$

- **Compute similarity**

- $Score = QK^T$

- **Scale**

- $Scaled = \frac{Score}{\sqrt{d_k}}$

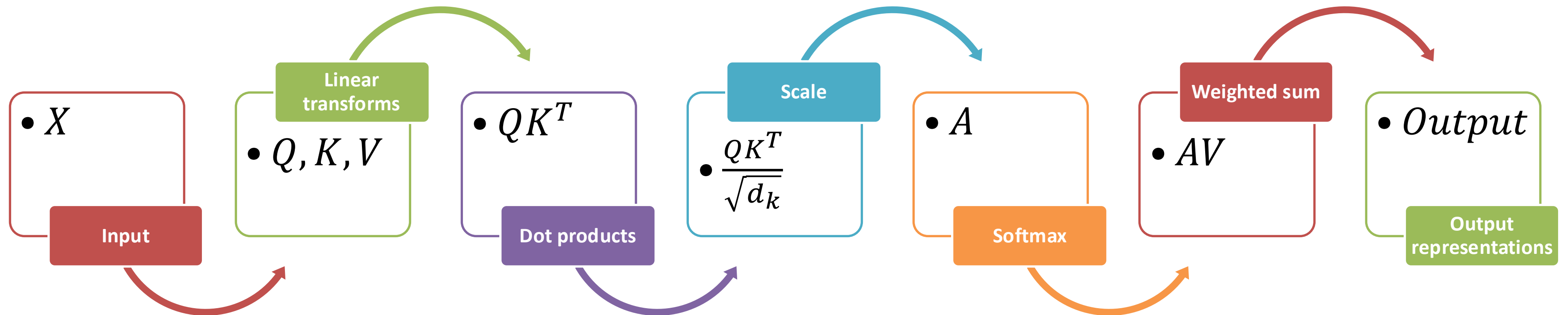
- **Softmax**

- $A = Softmax(Scaled)$

- **Output**

- $Output = AV$

Pipeline Diagram



Positional Encoding



Why is Positional Encoding Needed?

- Transformer attention has **no sense of order**.
- Example:
- "I love NLP"
- "NLP love I"
- If we only give embeddings, the Transformer sees them as a **set**, not a sequence.
- So we add **positional information**.

What is Positional Encoding?

- Positional encoding is a vector added to each word embedding:
- $X_{final} = X_{embedding} + PE$
- Where:
- $X_{embedding}$ = word embedding
- PE = positional encoding vector

Positional Encoding Formula (Sin/Cos)

- For position pos and dimension index i :

- **For even index (0,2,4,...)**

- $PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$

- **For odd index (1,3,5,...)**

- $PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$

- Where:

- pos = position of word (0,1,2,...)

- d = embedding dimension

Positional Encoding Formula (Sin/Cos)

- For position pos and dimension index i :

- **For even index (0,2,4,...)**

- $PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$

- **For odd index (1,3,5,...)**

- $PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$

- Where:

- pos = position of word (0,1,2,...)

- d = embedding dimension

Our Example Setup

- Sentence:
- $[I, Love, NLP]$
- Positions:
 - $I \rightarrow \text{position } 0$
 - $Love \rightarrow \text{position } 1$
 - $NLP \rightarrow \text{position } 2$
- Let embedding dimension:
 - $d = 4$
- So each word embedding has 4 values.

Step 1: Choose Word Embeddings

- Assume word embeddings are:

- **Embedding(I)**

- $X_0 = [1, 0, 1, 0]$

- **Embedding(Love)**

- $X_1 = [0, 1, 0, 1]$

- **Embedding(NLP)**

- $X_2 = [1, 1, 0, 0]$

- So, the embedding matrix is:

- $$X = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix}$$

- Shape:

- (3×4)

Step 2: Compute Positional Encoding

- **Values**
- We compute PE for each position:
 - pos = 0
 - pos = 1
 - pos = 2
- And for each dimension:
 - dim0
 - dim1
 - dim2
 - dim3
- **i = pair index (group index)**
- In the formula, they do not write sin/cos for every dimension separately.
- Instead they group dimensions into **pairs**:
- (dim 0, dim 1) is one pair
- (dim 2, dim 3) is second pair
- (dim 4, dim 5) third pair ... (if d is larger)
- Each pair uses one value of **i**.

Step 2: Compute Positional Encoding

- Why $i = 0$ gives dimensions 0 and 1?
- Because formula says:
- $PE(pos, 2i)$ and $PE(pos, 2i + 1)$
- Pair 1: $\text{dim0} = \sin$, $\text{dim1} = \cos$
- Pair 2: $\text{dim2} = \sin$, $\text{dim3} = \cos$
- Now plug values:
- If $i = 0$
 - Even dimension:
 - $2i = 2(0) = 0$
 - So that is **dimension 0**
 - Odd dimension:
 - $2i + 1 = 2(0) + 1 = 1$
 - So that is **dimension 1**
 - So $i=0$ corresponds to **dim 0** and **dim 1**
- If $i = 1$
 - Even dimension:
 - $2i = 2(1) = 2$
 - So that is **dimension 2**
 - Odd dimension:
 - $2i + 1 = 2(1) + 1 = 3$
 - So that is **dimension 3**
 - So $i=1$ corresponds to **dim 2** and **dim 3**
 - So in $d = 4$, possible i values?
 - Because total dimensions are 0,1,2,3
 - Pairs are:
 - Pair 1: $(0,1) \rightarrow i = 0$
 - Pair 2: $(2,3) \rightarrow i = 1$
 - So only:
 - $i = 0,1$



Step 3: Compute the Denominators

- We use:
- $10000^{\frac{2i}{d}}$
- Here $d = 4$
- So possible values of i are:
- **Pair 1: $i = 0$** $\rightarrow$ dimensions 0 and 1
- **Pair 2: $i = 1$** $\rightarrow$ dimensions 2 and 3
- **For $i = 0$:**
- $10000^{\frac{2(0)}{4}} = 10000^0 = 1$
- So divisor = 1.
- **For $i = 1$:**
- $10000^{\frac{2(1)}{4}} = 10000^{0.5} = \sqrt{10000} = 100$
- So divisor = 100.

Step 4: Compute PE for Each Position (pos = 0)

- **Position 0 (pos = 0)**

- **Dimension 0 (even)**

- $PE(0,0) = \sin\left(\frac{0}{1}\right) = \sin(0) = 0$

- **Dimension 1 (odd)**

- $PE(0,1) = \cos\left(\frac{0}{1}\right) = \cos(0) = 1$

- **Dimension 2 (even)**

- $PE(0,2) = \sin\left(\frac{0}{100}\right) = \sin(0) = 0$

- **Dimension 3 (odd)**

- $PE(0,3) = \cos\left(\frac{0}{100}\right) = \cos(0) = 1$

- So positional encoding for position 0:

- $PE_0 = [0, \quad 1, \quad 0, \quad 1]$

Step 4: Compute PE for Each Position (pos = 1)

- **Position 1 (pos = 1)**

- **Dimension 0 (even)**

- $PE(1,0) = \sin\left(\frac{1}{1}\right) = \sin(1)$

- $\sin(1) \approx 0.84147$

- $PE(1,0) = 0.84147$

- **Dimension 1 (odd)**

- $PE(1,1) = \cos\left(\frac{1}{1}\right) = \cos(1)$

- $\cos(1) \approx 0.54030$

- $PE(1,1) = 0.54030$

- **Dimension 2 (even)**

- $PE(1,2) = \sin\left(\frac{1}{100}\right) = \sin(0.01)$

- $\sin(0.01) \approx 0.00999983$

- $PE(1,2) = 0.00999983$

- **Dimension 3 (odd)**

- $PE(1,3) = \cos\left(\frac{1}{100}\right) = \cos(0.01)$

- $\cos(0.01) \approx 0.99995$

- $PE(1,3) = 0.99995$

- So positional encoding for position 1:

- $PE_1 = [0.84147, 0.54030, 0.00999983, 0.99995]$

Step 4: Compute PE for Each Position (pos = 2)

- **Position 2 (pos = 2)**

- **Dimension 0 (even)**

- $PE(2,0) = \sin\left(\frac{2}{1}\right) = \sin(2)$

- $\sin(2) \approx 0.90930$

- $PE(2,0) = 0.90930$

- **Dimension 1 (odd)**

- $PE(2,1) = \cos\left(\frac{2}{1}\right) = \cos(2)$

- $\cos(2) \approx -0.41615$

- $PE(2,1) = -0.41615$

- **Dimension 2 (even)**

- $PE(2,2) = \sin\left(\frac{2}{100}\right) = \sin(0.02)$

- $\sin(0.02) \approx 0.01999867$

- $PE(2,2) = 0.01999867$

- **Dimension 3 (odd)**

- $PE(2,3) = \cos\left(\frac{2}{100}\right) = \cos(0.02)$

- $\cos(0.02) \approx 0.99980$

- $PE(2,3) = 0.99980$

- So positional encoding for position 2:

- $PE_2 = [0.90930, -0.41615, 0.01999867, 0.99980]$

Step 5: Create the Full Positional

- **Encoding Matrix**
- Now combine PE vectors:
- PE

$$= \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0.84147 & 0.54030 & 0.00999983 & 0.99995 \\ 0.90930 & -0.41615 & 0.01999867 & 0.99980 \end{bmatrix}$$

- Shape:
- (3×4)

Step 6: Add Embeddings + Positional Encoding

- We compute:
- $X_{final} = X + PE$
- **For word 1: "I" (pos 0)**
 - $X_0 = [1, 0, 1, 0]$
 - $PE_0 = [0, 1, 0, 1]$
- **Add element-wise:**
 - $X_{final,0} = [1 + 0, 0 + 1, 1 + 0, 0 + 1]$
 - $X_{final,0} = [1, 1, 1, 1]$
- **For word 2: "Love" (pos 1)**
 - $X_1 = [0, 1, 0, 1]$
 - $PE_1 = [0.84147, 0.54030, 0.00999983, 0.99995]$
- **Add:**
 - $X_{final,1} = [0 + 0.84147, 1 + 0.54030, 0 + 0.00999983, 1 + 0.99995]$
 - $X_{final,1} = [0.84147, 1.54030, 0.00999983, 1.99995]$
- **For word 3: "NLP" (pos 2)**
 - $X_2 = [1, 1, 0, 0]$
 - $PE_2 = [0.90930, -0.41615, 0.01999867, 0.99980]$
- **Add:**
 - $X_{final,2} = [1 + 0.90930, 1 - 0.41615, 0 + 0.01999867, 0 + 0.99980]$
 - $X_{final,2} = [1.90930, 0.58385, 0.01999867, 0.99980]$

Final Output Matrix After Positional

- Encoding

- So:

- $X_{final} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0.84147 & 1.54030 & 0.009999983 & 1.99995 \\ 1.90930 & 0.58385 & 0.019999867 & 0.99980 \end{bmatrix}$

- This is what is fed into the Transformer attention block.

Full Table with Labels

Word Embeddings Matrix X

Word	pos	dim0	dim1	dim2	dim3
I	0	1	0	1	0
Love	1	0	1	0	1
NLP	2	1	1	0	0

Positional Encoding Matrix PE

Word	pos	PE0	PE1	PE2	PE3
I	0	0.00000	1.00000	0.00000	1.00000
Love	1	0.84147	0.54030	0.00999983	0.99995
NLP	2	0.90930	-0.41615	0.01999867	0.99980

Final Transformer Input $X_{final} = X + PE$

Word	pos	final0	final1	final2	final3
I	0	1.00000	1.00000	1.00000	1.00000
Love	1	0.84147	1.54030	0.00999983	1.99995
NLP	2	1.90930	0.58385	0.01999867	0.99980

Transformer Model Visually Explained

- <https://poloclub.github.io/transformer-explainer/>

